Hirexp Ai

Resume Screening with AI-Powered Interview using Interactive AI Avatars

Name of the Team members	Roles
Satish	Project Lead and Core Backend
Sravani	AI/NLP
Mounika	Frontend and Deployment
Sumanjali	QA, Data and Documentation

1. Introduction

1.1 Project Overview

Hirexp Ai is an enterprise-style recruitment platform that automates two of the most time-consuming stages of hiring: screening large volumes of resumes, and conducting first-round interviews. A recruiter creates a job requisition, uploads resumes in bulk, and receives a ranked, evidence-backed list of candidates. Shortlisted candidates are invited, through a secure single-use link, to a virtual interview hosted by an animated AI avatar. Their answers are scored against transparent rubrics, and the results are compiled into a candidate dossier for a human recruiter to review.

The system is built in Python with a Streamlit web interface, SQLAlchemy for persistence, PyMuPDF for document parsing, scikit-learn for text similarity, and the Anthropic Claude API for language

understanding. A deterministic offline (mock) mode allows the entire application to run without an API key.

1.2 Problem Statement

Recruiters receive hundreds of resumes per opening. Manual screening is slow, inconsistent between reviewers, and prone to unconscious bias. First-round interviews add further scheduling overhead. Keyword-only screening tools miss qualified candidates whose wording differs from the job description, and they rarely explain why a candidate was ranked the way they were.

1.3 Objectives

- Process resumes in bulk, quickly and reliably, with duplicate detection and failure recovery.
- Score every candidate with a multi-metric, explainable model that cites evidence from the resume.
- Keep a human recruiter in control: the AI recommends, the recruiter decides, and overrides are recorded.
- Deliver an engaging, resume-grounded interview experience through an interactive AI avatar with voice output and typed or spoken answers.
- Exclude protected characteristics (name, age, gender, race, religion, marital status) from all scoring.
- Provide security, role-based access control, audit trails and GDPR-style data deletion.

1.4 Scope

In scope: multi-company campaigns, job requisitions, bulk PDF/DOCX ingestion, hybrid scoring, shortlist governance, secure invitations, avatar-led technical and HR interviews, rubric-based evaluation, reports, analytics, exports and privacy tooling.

Out of scope OCR for scanned resumes and automatic email delivery of invitations.

2. Key Features

- **Campaigns and jobs:** multiple companies and isolated recruitment campaigns; requisitions with required and preferred skills, mandatory qualifications, minimum experience, interview topics, stages, duration and custom scoring weights; AI extraction of requirements from a job description.
- **Bulk resume processing:** PyMuPDF text extraction; health checks for encrypted, corrupt, empty and scanned PDFs; DOCX support; SHA-256 duplicate detection; thread-pool concurrency; live progress display; one-click retry of failed files.
- **Screening engine:** keyword coverage, TF-IDF cosine similarity, experience match and Claude semantic analysis combined into a weighted 0 to 100 score, with verbatim evidence excerpts.
- **Shortlist governance:** filter, sort and search; human approval with recorded override reasons; bulk approval above a threshold.
- **Secure invitations:** HMAC-signed, single-use tokens that expire after 72 hours by default.
- **Virtual interview room:** an animated avatar with idle, speaking, listening and thinking states; the question read aloud by the browser; answers given by typing or by voice converted to text; Technical and HR tracks.
- **Evaluation:** 100-point transparent rubrics, concept coverage, feedback, improvement suggestions and contextual follow-up questions.
- **Reports and analytics:** printable HTML dossiers, CSV and Excel export, and Plotly funnel and skill-gap charts.

- **Security and privacy:** PBKDF2 password hashing, role-based access, company isolation, audit events, consent capture, right to erasure and retention purge.

3. Technology Stack

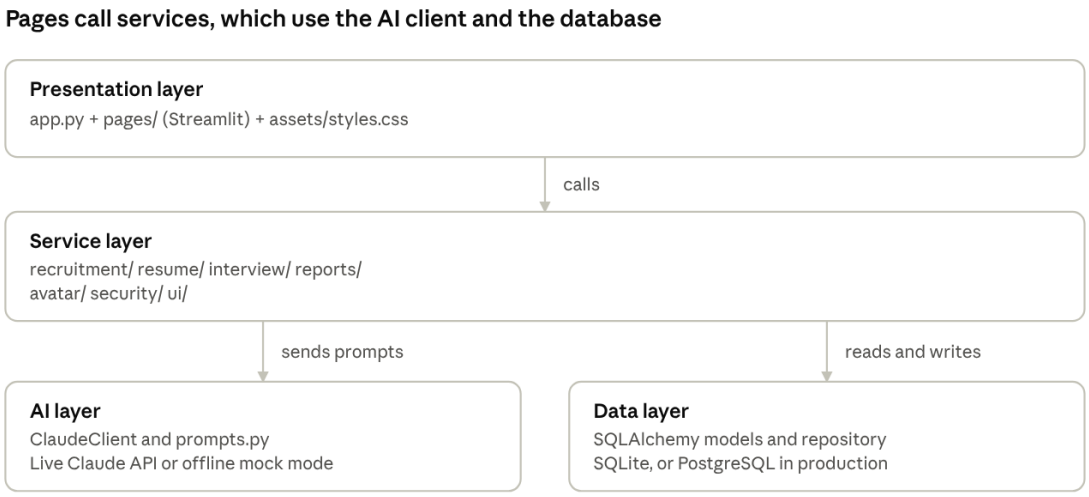
Layer	Technology	Purpose
Language	Python 3.11+	Application code
Web UI	Streamlit 1.35+, custom CSS, HTML5/SVG, Plotly	Multi-page recruiter and candidate interfaces, charts
Data modelling	Pydantic v2	Validated resume and screening schemas, settings
Database	SQLAlchemy 2.0 with SQLite (PostgreSQL-ready)	ORM and persistence
Document parsing	PyMuPDF; optional python-docx or XML fallback	PDF and DOCX text extraction
NLP	scikit-learn (TF-IDF, cosine similarity)	Lexical similarity
Generative AI	Anthropic Claude API (default model claude-3-5-sonnet-20241022)	Extraction, semantic matching, question generation, answer evaluation
Reporting	pandas, ReportLab, Plotly	Exports, synthetic PDF generation, charts

Layer	Technology	Purpose
Browser APIs	Web Speech Synthesis	Avatar voice and spoken answers
Quality	pytest, Ruff (line length 120, target py311)	Automated tests and linting

4. System Architecture

4.1 Layered Design

The application follows a layered architecture. Streamlit pages contain only presentation logic and call into service classes; services work with the ORM models; the AI client is isolated behind a single class so that live and mock modes are interchangeable.



layered architecture · 4 layers, 3 calls

Read top down: pages call services, and services use the AI client and the data layer.

4.2 End-to-End Workflow

1. The recruiter signs in and creates a campaign and a job requisition.
2. Resumes are uploaded in bulk. Each file is validated, hashed, de-duplicated, text-extracted, parsed into a structured profile and scored.
3. The recruiter reviews ranked results and evidence, then shortlists or rejects candidates.
4. Approved candidates receive a single-use interview token.
5. The candidate opens the Virtual Interview Room, gives explicit consent and answers resume-grounded questions put by the avatar.
6. Each answer is scored against the rubric; follow-up questions are generated; the session score is the mean of question scores.
7. A dossier combining screening evidence and interview performance is produced for the recruiter's final decision.

4.3 Project Structure

- **app.py:** the home page with KPIs, pipeline overview and recent activity.
- **config.py:** the central Settings object (database, security, AI, concurrency, weights, retention).
- **pages/:** eleven Streamlit pages, from sign-up and the dashboard to bulk upload, results, shortlist, interviews, reports, analytics and settings.
- **src/ai/:** ClaudeClient and the prompt templates.
- **src/resume/:** PDF parser, resume extractor, schemas, skill matcher, screening engine and batch processor.
- **src/interview/:** interview engine, dynamic and static question generators, answer evaluator, rubrics and resume profile.

- **src/recruitment/:** campaign, shortlist and invitation services.
- **src/reports/:** candidate dossier and interview report generators.
- **src/avatar/:** avatar component and speech service.
- **src/security/:** authentication, authorization, session manager and privacy manager.
- **src/database/:** ORM models, engine, initialisation and seeding, and repository helpers.
- **src/ui/:** shared layout helpers, theme and chart styling.
- **data/, scripts/, tests/:** the database and synthetic resumes, the throughput benchmark, and eight pytest modules.
- **AUDIT_REPORT.md:** the security audit and remediation record.

The repository archive also contains a folder named gi-disease-classification. It is a separate deep-learning project for gastrointestinal image classification and is not part of Hirexp Ai; it is not covered in this document.

5. Functional Modules

5.1 Configuration (config.py)

A Pydantic Settings object reads environment variables and defines the following defaults.

- **Identity:** APP_NAME is Hirexp Ai.
- **Database:** DATABASE_URL defaults to a local SQLite file.
- **Security:** SECRET_KEY comes from the environment, otherwise a random 48-byte token is generated at start-up; invitation tokens expire after 72 hours and recruiter sessions after 24 hours.
- **Uploads and concurrency:** 25 MB per file, .pdf only, 6 worker threads and chunks of 25 files.
- **Scoring:** four default weights of 0.25 each and a shortlist threshold of 70.

- **Privacy:** a 90-day data retention period with automatic purge allowed.

5.2 AI Layer (src/ai)

ClaudeClient is the single gateway to the Anthropic API. It calls the model with temperature 0.2, strips Markdown code fences from replies, and parses JSON defensively. If the API key is missing, the client cannot be initialised, or a call fails, it silently falls back to a deterministic heuristic implementation, so the application never stops working.

It exposes five operations, each backed by a prompt template in prompts.py:

- **extract_job_requirements:** Claude returns required and preferred skills, mandatory qualifications, education, minimum years and technical subjects. The fallback matches the job text against a built-in catalogue of about 37 technologies.
- **extract_resume_data:** Claude parses the first 6,000 characters into structured JSON with evidence quotes, and the prompt forbids inferring protected attributes or inventing data. The fallback uses regular expressions for email and phone, the first line as the name, and catalogue-based skill detection.
- **evaluate_semantic_match:** Claude judges job-to-candidate fit and returns a score with a rationale. The fallback score is 20 plus 70 times the share of required skills matched.
- **generate_interview_questions:** Claude returns structured questions with expected concepts and a rubric. The fallback uses fixed Technical and HR question sets.
- **evaluate_interview_answer:** Claude scores the answer against the rubric. The fallback scores from concept coverage and answer length.

5.3 Resume Ingestion (src/resume)

PDFParser opens documents with PyMuPDF and returns text, page count and health flags. It reports empty files, corrupt files, encrypted files and zero-page documents as errors. A document with fewer than 50 characters of selectable text but embedded images is flagged as scanned. DOCX files are read with python-docx when installed, otherwise by reading word/document.xml directly from the archive.

ResumeExtractor sanitises the text, asks the AI layer for structured data, and validates the result with Pydantic models (StructuredResume with education, work experience, project, certification and skill-evidence fields). If validation fails, a safe minimal profile is returned.

BatchResumeProcessor processes uploads concurrently with a ThreadPoolExecutor. Each file runs in its own database transaction so one failure never aborts the batch. For each file it validates the document, computes a SHA-256 hash, checks for an existing copy in the same campaign (marking duplicates), parses the resume and screens it. A progress object records totals, successes, failures, duplicates, elapsed time and throughput. retry_failed_resumes re-processes previously failed files from disk.

ScreeningEngine ties the steps together: it parses the resume, creates or updates the CandidateProfile, stores ParsedResumeData, calls the matcher against the job, stores a ScreeningResult and marks the resume as Processed.

5.4 Screening and Scoring (SkillMatcher)

SkillMatcher computes four independent scores, each on a 0–100 scale.

- **Keyword coverage:** the percentage of the job's required skills found (case-insensitive) in the candidate's extracted skills; matched and missing lists are kept.

- **Text similarity:** TF-IDF vectors (English stop-words, 1 to 2-grams, up to 5,000 features) of the job description and the resume, with cosine similarity multiplied by 100.
- **Experience:** the ratio of candidate years to required years. 1.2 or more scores 100, 1.0 or more scores 90, 0.75 or more scores 75, 0.5 or more scores 55, and anything lower scores 30.
- **Semantic:** the Claude semantic judgement, or the offline fallback.

The overall score is the weighted average of the four metrics using the job's own weights (default 0.25 each, normalised by the total weight):

```
overall = (kw × w_kw + sim × w_sim + exp × w_exp + sem × w_sem) / (w_kw + w_sim + w_exp + w_sem)
where as:
kw = (required skills found in resume ÷ total required skills) × 100
```

A five-factor variant that adds preferred-skill coverage and mandatory-qualification score is also supported by the matcher.

The recommendation is then derived as follows:

- Overall score at or above the threshold (default 70) with all mandatory criteria passed: **Meets Configured Criteria**.
- Overall score within 15 points below the threshold: **Needs Manual Review**.
- Anything lower: **Does Not Meet Configured Criteria**.

For every matched skill the engine stores a short excerpt from the resume as evidence, so the recruiter can verify each claim. Names, age, gender and other protected attributes play no part in any metric.

5.5 Recruitment Services (src/recruitment)

- **CampaignService** creates campaigns and jobs and produces dashboard metrics (funnel counts, screening health, pipeline stages).
- **ShortlistService** returns the filterable candidate list, records a recruiter decision with notes and an override reason, and performs bulk approval.
- **InvitationService** creates invitations and validates tokens. Validation rejects unknown, expired and already-used tokens, which gives replay protection.

Recruiter decisions follow the lifecycle Pending → Shortlisted / Rejected → Invited → Interviewed → Offer Extended → Hired.

5.6 Virtual Interview Engine (src/interview)

The InterviewEngine coordinates a complete interview session.

Resume-grounded profile. Before the interview starts, the engine builds a NormalizedResumeProfile from the candidate's parsed resume: skills, programming languages, frameworks and tools, projects, work experience, education, certifications and achievements. If the profile is empty, safe defaults are used so the session can still run.

Topic distribution. DynamicQuestionGenerator allocates the questions of a session across five categories. If the resume has no certifications or internships, or no projects, the unused share is redistributed to the remaining categories and the quotas are re-normalised; the order is then shuffled.

- Technical skills: 30%
- Projects: 30%
- HR and behavioural: 20%
- Certifications or internships: 10%

- Problem-solving and architecture: 10%

Question generation. For each slot the generator picks a topic from the resume (avoiding projects already asked about), builds a prompt for Claude and, if the AI is unavailable, falls back to randomised templates that are still rooted in the candidate's own skills and projects. Questions avoid repeating earlier ones and carry expected concepts and a rubric. The default session length is five questions at Intermediate difficulty.

Session lifecycle.

1. start_session checks for an existing in-progress session for the same candidate, job and stage and resumes it; otherwise it records the candidate's consent with a timestamp and creates the session and its questions.
2. submit_answer sanitises and evaluates the answer, stores the score, feedback, correct and missing concepts, improvement suggestions and rubric breakdown, generates a contextual follow-up question, and advances the question index.
3. When the last question is answered the session is finalised: status becomes Completed, the overall interview score is the mean of the question scores, the invitation is marked used, and the candidate's recruiter decision moves to Interviewed.

Transparent rubrics. Every rubric totals exactly 100 points and is validated in code.

- **Technical track:** technical correctness 40, reasoning and explanation 25, practical application 20, communication and structure 15.
- **HR track (STAR):** relevance 30, structure and clarity (Situation, Task, Action, Result) 25, specific examples 25, communication 20.

An empty answer receives a score of zero with a prompt to try again, and the AnswerEvaluator sanitises every answer before it reaches the model.

5.7 Interactive AI Avatar (src/avatar)

The avatar is what distinguishes the interview from a plain questionnaire.

- **avatar_component.py** renders the interviewer image inside an HTML component with four visual states: idle, speaking, listening and thinking. A state badge and CSS animation show what the interviewer is doing, and in the speaking state the browser's speech synthesis reads the question aloud.
- **speech_service.py** supplies browser-based controls to replay or pause the spoken question and a voice-to-text option for spoken answers. Typed answers are always available as an alternative.
- Each answer is stored as text together with its answer mode (voice converted to text, or typed) for transparency.

The avatar is an animated image. Voice features rely on the candidate's browser, so their availability depends on the browser in use.

5.8 Reports and Exports (src/reports)

- **ReportGenerator** builds a printable, downloadable HTML candidate dossier and exports candidate tables to a pandas DataFrame, CSV bytes and Excel bytes.
- **InterviewReportService** produces an interview performance report (per-question scores, rubric breakdown, strengths, gaps and recommendations) and renders it as HTML.
- All candidate-supplied text placed in HTML is escaped to prevent script injection.

6. Security and Privacy

6.1 Authentication and Sessions

- Passwords are hashed with PBKDF2-HMAC-SHA256 using a per-user salt and 100,000 iterations, stored in a salt, iterations and hash format. Verification uses a constant-time comparison and returns false for any malformed hash. There is no fallback or demo password path.
- SessionManager keeps an AuthUser (id, name, email, role, company, login time) in the Streamlit session, enforces the 24-hour expiry, shows a profile banner with role in the sidebar, handles sign-out, and provides require_auth(allowed_roles) to protect pages.
- Candidates never sign in to the recruiter portal; they reach only the Virtual Interview Room through a valid invitation token.

6.2 Invitation Tokens

Tokens have the form `hs-inv-<32 hex characters>-<24-character signature>`. The signature is an HMAC-SHA256, keyed with the application SECRET_KEY, over the candidate, job, random bytes and a timestamp. Tokens are unique, expire after 72 hours by default, and are marked used when the interview completes.

6.3 Role-Based Access Control

AuthorizationService maps each role to a set of permissions and halts a page with an access-denied message when a permission is missing. Queries are scoped by company so each organisation sees only its own campaigns and candidates.

Permission	Admin	Recruiter	Hiring manager	Candidate
View dashboard, screening, reports, analytics	Yes	Yes	Yes	No
Update decision	Yes	Yes	Yes	No
View interview records	Yes	Yes	Yes	No
Create and edit jobs	Yes	Yes	No	No
Upload resumes, manage shortlist, invite candidates	Yes	Yes	No	No
View audit logs	Yes	Yes	No	No
Delete jobs, manage users, privacy purge	Yes	No	No	No
Attend interview	Yes	No	No	Yes

6.4 Audit Trail

Important actions (decisions, interview creation, deletions) are written to the audit_events table with the actor, company, entity, safe metadata and timestamp. Full resume contents are deliberately never logged.

6.5 Input Protection

sanitize_input_text removes null bytes and replaces well-known prompt-injection phrases (for example "Ignore all previous instructions") with a filtered marker before resume text or interview answers reach the AI. All dynamic fields rendered as HTML are escaped.

6.6 Privacy and Compliance

- **Explicit consent.** The interview consent checkbox is unchecked by default and the consent time is stored.
- **Privacy notice.** PrivacyManager publishes a notice covering purposes, the protected-characteristics policy, retention and candidate rights (access, rectification, erasure and human override).
- **Right to erasure.** delete_candidate_data permanently removes the candidate profile, resumes with their stored files, invitations and interview sessions.
- **Retention.** purge_expired_records removes records older than the retention period (90 days by default).

6.7 Security Audit Outcomes

AUDIT_REPORT.md records the findings of a security review and their resolution.

- **SEC-01 (critical):** a demo password fallback accepted malformed hashes. The fallback was removed, so invalid hashes always fail.
- **SEC-02 (critical):** the default secret key was predictable. A random key is now generated unless SECRET_KEY is set.
- **SEC-03 (high):** active tokens were exposed in the interview room. The token dropdown and its database query were removed.
- **SEC-04 (high):** the consent checkbox was pre-ticked. It now defaults to unchecked.
- **SEC-05 (high):** candidate data was rendered unescaped in HTML. html.escape is now applied to dynamic fields.
- **SEC-06 (high):** there was no recruiter authentication or role-based access. SessionManager, roles and company isolation were added.
- **SEC-07 (medium):** there were no security regression tests. tests/test_security_regressions.py was added.

7. Database Design

7.1 Overview

The schema is defined with SQLAlchemy 2.0 declarative models and created automatically on start-up by init_db(), which also seeds a demonstration company, an administrator, a recruiter, a campaign and a sample Senior Python Backend Engineer job when the database is empty. Primary keys are UUID strings. JSON columns hold lists and dictionaries such as skills and evidence. The engine is configured with check_same_thread=False so SQLite works under Streamlit's threads.

7.2 Tables

Table	Purpose	Key fields
companies	Tenant organisations	name (unique), domain, industry
recruiters	Staff accounts	company_id, email (unique), password_hash, role, is_active
recruitment_campaigns	Hiring drives	company_id, title, status, target_hires
jobs	Requisitions	campaign_id, required and preferred skills, mandatory qualifications, min_years_experience, interview stages and duration, four weights, shortlist_threshold

Table	Purpose	Key fields
candidate_profiles	Candidate identity and links	name, email, phone, linkedin, github, years_experience
resume_documents	Uploaded files	job_id, candidate_id, sha256_hash, page_count, scanned/encrypted/corrupt/duplicate flags, processing_status, raw_text
parsed_resume_data	Structured resume	skills, education, work_experience, projects, certifications, skill_evidence
screening_results	Scores and decisions	overall and four metric scores, matched and missing skills, evidence, recommendation, recruiter decision, notes and override reason
interview_invitations	Secure invites	candidate_id, job_id, token (unique), stage, expires_at, is_used
interview_sessions	Interviews	stage, status, current_question_index, total_questions, overall_interview_score, consent and timestamps
interview_questions	Questions and answers	question text and type, category, expected concepts, rubric, candidate answer, answer mode,

Table	Purpose	Key fields
		score, feedback, rubric scores, follow-up
audit_events	Security audit log	action, entity, actor, details, IP, time
data_retention_records	Purge scheduling	entity, scheduled purge date, purged flag

7.3 Relationships

Company has many recruiters and campaigns. A campaign has many jobs and resumes. A job has many resumes and invitations. A candidate has many resumes, invitations and interview sessions. Each resume has at most one parsed-data record and one screening result. An interview session has many questions. Deleting a parent cascades to its dependent records.

7.4 Status Enumerations

- **UserRole:** admin, recruiter, hiring_manager, candidate.
- **ProcessingStatus:** Uploaded, Queued, Processing, Processed, Needs Manual Review, Meets Configured Criteria, Does Not Meet Configured Criteria, Failed, Duplicate.
- **RecruiterDecision:** Pending, Shortlisted, Rejected, Invited, Interviewed, Offer Extended, Hired.
- **InterviewStage:** Technical, HR, Comprehensive.
- **InterviewStatus:** Not Scheduled, Invitation Sent, In Progress, Completed, Expired, Cancelled.

8. User Interface

The interface uses a dark navy and indigo design system defined in assets/styles.css, with reusable helpers in src/ui/layout.py (header and breadcrumbs, KPI cards, badges, empty states, score badges and Plotly theming). Each page checks the user's role through setup_sidebar_context.

- **Home (app.py):** command centre with a welcome message, KPIs and recent activity.
- **0 Sign Up:** register a recruiter account for an organisation.
- **1 Recruitment Dashboard:** funnel conversion, screening health and pipeline overview.
- **2 Create Job:** create a job requisition (with AI requirement extraction and weights) or a new campaign.
- **3 Bulk Resume Upload:** multi-file upload with live progress, duplicate and failure reporting, and retry.
- **4 Screening Results:** ranked candidates, score breakdown, matched and missing skills, and evidence excerpts.
- **5 Shortlist Management:** review the proposed shortlist, approve or reject with reasons, and issue invitation tokens.
- **6 Interview Management:** session progress, transcripts, rubric evaluations and timestamps.
- **7 Virtual Interview:** the candidate-facing room with token entry or a custom resume test, consent, avatar, answers, follow-ups and report.
- **8 Candidate Reports:** multi-stage dossiers, downloads, and CSV or Excel export.
- **9 Analytics:** score distributions, skill bottlenecks and funnel velocity charts.
- **10 Settings and Privacy:** profile, privacy and erasure tools, AI engine status and the audit log.

9. Data, Testing and Quality Assurance

9.1 Synthetic Dataset

data/synthetic_generator.py uses ReportLab to produce realistic synthetic PDF resumes. The bundled set contains 100 numbered resumes spread across four tracks (AI, DevOps, backend, general) and three seniority levels (junior, mid, senior), plus five deliberate duplicate copies used to verify SHA-256 duplicate detection.

9.2 Automated Tests

- **test_pdf_parser.py:** PDF text extraction and document health detection.
- **test_screening.py:** keyword, similarity, experience and composite scoring.
- **test_bulk_processing.py:** batch ingestion, duplicates and progress reporting.
- **test_database.py:** schema creation and persistence.
- **test_interview.py:** interview session flow and rubrics.
- **test_dynamic_interview.py:** topic distribution and resume-grounded questions.
- **test_authorization.py:** role permissions and company scoping.
- **test_security_regressions.py:** password verification, malformed hash rejection and access control.

The security audit recorded 15 passing tests at the time of review. The suite has since gained further modules, and the test results were not re-run while preparing this document; run pytest tests -v to obtain current results.

9.3 Performance Benchmark

scripts/benchmark_bulk.py measures ingestion throughput. The project README reports the following result for 50 PDF resumes processed with 6 worker threads.

- Batch size: 50 resumes
- Total time: 2.41 seconds
- Average per resume: 48.2 ms
- Throughput: 20.76 resumes per second
- Success rate: 98.0%, with 2.0% detected as duplicates
- Failure rate: 0.0%

9.4 Code Quality

Ruff is configured for pycodestyle errors, pyflakes, import sorting and warnings with a 120-character line length, and pytest is configured to discover tests under tests/ with the project root on the path.

10. Installation and Deployment

10.1 Prerequisites

Python 3.11 or later, pip, and optionally an Anthropic API key. No API key is needed for offline mode.

10.2 Setup Steps

1. Obtain the project (git clone of the repository or extract the archive) and open a terminal in the project folder.
2. Create and activate a virtual environment: python -m venv venv, then venv\Scripts\activate on Windows or source venv/bin/activate on Linux and macOS.

3. Install dependencies: pip install -r requirements.txt (streamlit, pydantic, sqlalchemy, pymupdf, scikit-learn, anthropic, plotly, pandas, pillow, reportlab, pytest, ruff).
4. Copy .env.example to .env and set the variables listed below.
5. Optional: generate sample resumes with python data/synthetic_generator.py.
6. Start the application with streamlit run app.py and open http://localhost:8501.

10.3 Environment Variables

- **ANTHROPIC_API_KEY:** API key for live Claude features; leave empty for offline mode.
- **CLAUDE_MODEL:** model name (default claude-3-5-sonnet-20241022).
- **USE_MOCK_AI:** set to true to force the deterministic offline engine.
- **DATABASE_URL:** SQLAlchemy URL; SQLite by default, PostgreSQL supported.
- **SECRET_KEY:** secret used to sign invitation tokens; set a fixed value in production.
- **MAX_WORKERS:** threads used for bulk processing (default 6).

10.4 Deployment Notes

- Set a strong, persistent SECRET_KEY; otherwise a new random key is generated at each start and previously issued tokens will no longer validate.
- Change the seeded demonstration passwords (Appendix A) before any real use.
- For multi-user deployments, point DATABASE_URL at PostgreSQL and store uploads on persistent storage.

- Serve the Streamlit app behind HTTPS, since voice input in browsers requires a secure context.

---

11. User Guide

11.1 Recruiter

1. Sign in (or create an account on the Sign Up page).
2. On Create Job, create a campaign and a job. Paste a job description and use AI extraction to fill the skills, qualifications and subjects, then adjust the weights and shortlist threshold.
3. On Bulk Resume Upload, select the campaign and job, then upload PDF or DOCX resumes. Watch progress; use retry for any failed file.
4. On Screening Results, review scores, matched and missing skills and the quoted evidence.
5. On Shortlist Management, approve or reject candidates, record override reasons where you disagree with the AI, and generate invitation tokens.
6. Send each token to the candidate. After the interview, review sessions on Interview Management and dossiers on Candidate Reports.
7. Use Analytics for funnel and skill-gap insight, and export CSV or Excel when needed.

11.2 Candidate

1. Open the Virtual Interview Room and enter the invitation token.
2. Read the privacy notice and tick the consent box.
3. Listen to each question from the avatar (or replay it).

4. Answer by typing, or by voice if you prefer; spoken answers are converted to text.
5. Review the follow-up and feedback, then finish all questions to complete the interview.

11.3 Administrator

Administrators have every permission. They manage the audit log, run privacy purges and carry out right-to-erasure requests from Settings and Privacy.

12. Limitations and Future Enhancements

12.1 Current Limitations

- Scanned PDFs are detected and flagged but are not read, because OCR is not implemented.
- Offline mode relies on heuristics and contains placeholder values (for example a fixed years-of-experience figure and a sample education entry), so offline scores are suitable for demonstration and testing only.
- Keyword coverage uses exact, case-insensitive skill matching; synonyms are only credited through the semantic score.
- Mandatory qualifications are checked explicitly only for years of experience; other criteria are treated as met unless the semantic evaluation indicates otherwise.
- The prompt-injection sanitiser matches a fixed list of phrases and is not a complete defence.
- Invitation tokens are generated in the application but are not emailed automatically.
- The avatar is a 2D animated image.
- Voice answers depend on browser support for speech.

- SQLite suits a single machine; larger deployments need PostgreSQL.

12.2 Future Enhancements

- OCR for scanned resumes and richer layout-aware parsing.
- Embedding-based skill matching with synonym and skills-taxonomy support.
- Automatic email and calendar delivery of invitations.
- Multilingual interviews.
- Single sign-on, container-based deployment and continuous integration.
- Fairness reporting and periodic bias audits of scoring outcomes.

13. Team and Responsibilities

Member	Role	Focus areas
Satish	Project Lead and Core Backend	Project planning and coordination; database models and repository; campaign, shortlist and invitation services; batch processing; security, sessions and RBAC
Sravani	AI/NLP	Claude client and prompt design; resume and job extraction; TF-IDF and semantic matching; dynamic question generation; answer evaluation and rubrics
Sumanjali	Frontend and Deployment	Streamlit pages and navigation; styling and layout system; avatar and speech components; charts;

Member	Role	Focus areas
		configuration and deployment
Mounika	QA, Data and Documentation	Automated tests and regression checks; synthetic resume generation; benchmarking; audit verification; project documentation

14. Conclusion

Hirexp Ai demonstrates how generative AI and classical NLP can be combined, with a human always in the loop, to make high-volume hiring faster and more consistent. Its hybrid, evidence-backed scoring explains each ranking; its avatar-led interview gives candidates a more engaging first round; and its security, consent and privacy controls address the main risks of using AI in recruitment. The offline mode and synthetic dataset make the system easy to demonstrate and test, and the modular design leaves clear room for the enhancements listed above.

Appendix A. Seeded Demonstration Accounts

On first start the database is seeded with the following accounts, as documented in AUDIT_REPORT.md. They exist for testing only and must be changed or removed before real use.

- **Administrator:** admin@hiresphere.ai, password AdminPass2026!
- **Recruiter:** recruiter@hiresphere.ai, password RecruiterPass2026!

Appendix B. Glossary

- **TF-IDF:** term frequency–inverse document frequency, a weighting of words by how distinctive they are in a document.
- **Cosine similarity:** a measure of the angle between two text vectors, used as a similarity score.
- **RBAC:** role-based access control.
- **HMAC:** hash-based message authentication code, used to sign tokens.
- **PBKDF2:** a key derivation function used to hash passwords.
- **STAR:** Situation, Task, Action, Result, a structure for behavioural answers.
- **Requisition:** a formal job opening with its criteria.
- **Mock mode:** deterministic offline mode that replaces calls to the Claude API.